PROYECTO DE AULA · TEORÍA DE LA COMPUTACIÓN

Avance 3
Metodología

Sistema de préstamo de herramientas basado en autómatas finitos y verificación visual

Documento de trabajo académico

Juan Diego Torres Mola

Ingeniería de Sistemas

Universidad de Pamplona

Docente: Mercy Lucero Gómez Rincón

Cúcuta, Norte de Santander

Avance 3 — VI. Metodología

Propósito y alcance del apartado

Esta metodología explica cómo se analiza, diseña, construye y evalúa el proyecto **Sistema de préstamo de herramientas basado en autómatas finitos y verificación visual**, conforme al título común de `docs/avances/BASE_PROYECTO.md`. Se organiza en las cinco fases solicitadas por la guía del Avance 3, conservada en `docs/avances/fuentes/guia_avance3.md`: análisis del problema, diseño de la solución, implementación o construcción, pruebas y validación, y documentación y presentación.

El documento describe el proyecto que existe en el repositorio; no reconstruye fechas ni atribuye actividades a personas sin evidencia. Para evitar confundir capacidades del software con resultados experimentales, se emplean tres estados:

- **Implementado:** existe código, un documento o una evidencia identificable en el repositorio.
- **Documentado como ejecutado:** una prueba figura como realizada en `docs/avances/BASE_PROYECTO.md`, `docs/VALIDACION.md` o `docs/SIMULADOR.md`; su mención aquí no equivale a una nueva ejecución propia.
- **Propuesto o pendiente:** requiere intervención del equipo, dispositivos reales, autorización institucional o integración académica posterior.

El alcance actual es un prototipo académico local para **una unidad de martillo**, con cámara real cuando se conecta, puerta y sensores simulados, persistencia SQLite, portal de control, monitor de solo lectura y un simulador independiente. No incluye Arduino, puerta física ni sensores físicos. El reconocimiento todavía no puede validarse con datos propios porque no hay dataset ni `models/herramientas.pt` disponibles en el árbol consultado; esta ausencia también está registrada en `docs/ESTADO.md` y `docs/VALIDACION.md`.

1. Enfoque de trabajo y tipo de proyecto

1.1 Enfoque

El proyecto adopta un enfoque de **investigación aplicada, iterativo y con evaluación experimental del prototipo**. Es aplicado porque utiliza autómatas finitos, desarrollo web y clasificación de imágenes para abordar una necesidad concreta de control de préstamos. Es iterativo porque los requisitos, los modelos de estados, la implementación y las pruebas se revisan como un conjunto: un rechazo o una incidencia permite corregir la regla, la interfaz o el procedimiento y repetir el caso. Incorpora evaluación experimental porque el comportamiento debe comprobarse con casos controlados y, posteriormente, con cámara, herramientas y condiciones reales documentadas.

La unidad de análisis es el **ciclo de préstamo y devolución de una unidad de martillo**, no el inventario completo de una institución. Se estudian tres niveles que no deben mezclarse:

1. **Nivel formal:** estados, eventos, transiciones y trazas aceptadas por los autómatas.
2. **Nivel de software:** autorización, persistencia, concurrencia, interfaz, historial y recuperación.
3. **Nivel operativo:** observaciones de cámara, calidad del dataset, comportamiento temporal y ejecución del procedimiento con dispositivos reales.

La puerta y los sensores simulados permiten estudiar la lógica sin atribuirle al prototipo capacidades de custodia física. El simulador visual sirve para explicar y ejercitar escenarios, pero no produce evidencia de reconocimiento ni modifica la base de préstamos del módulo principal (`docs/SIMULADOR.md`).

1.2 Justificación del enfoque

La necesidad combina reglas discretas —por ejemplo, no retirar sin reserva ni registrar dos préstamos activos— con observaciones inciertas procedentes de imágenes. Por esa razón no basta con entrenar un clasificador ni con programar formularios. El autómata ofrece una autoridad explícita para decidir qué evento es admisible; la cámara solo aporta observaciones, y el motor exige que sean recientes, estables y coherentes con la operación (`app/automata.py`, `app/engine.py`). Esta separación permite evaluar de manera independiente la corrección lógica y el desempeño visual.

El desarrollo incremental es adecuado porque existe una base funcional que puede probarse antes de construir hardware. Primero se delimita el proceso y se formalizan las reglas; luego se implementa el controlador y se verifican respuestas esperadas; finalmente se incorporan datos visuales propios y ciclos integrales. La secuencia reduce el riesgo de atribuir a una caja física o a un modelo resultados obtenidos únicamente con observaciones controladas.

1.3 Información necesaria, fuentes y tratamiento

Información	Fuente	Técnica de obtención	Uso metodológico
Proceso actual, actores, reglas y excepciones del laboratorio	Laboratorio e institución que el equipo seleccione	Observación directa y revisión de documentos internos autorizados; una entrevista solo si el equipo la programa y deja constancia real	Confirmar el problema, los requisitos y la correspondencia de los roles
Estados, eventos y restricciones del préstamo	`docs/INFORME.md`, `app/automata.py`, `app/engine.py`	Análisis documental y modelado formal	Definir transiciones válidas, rechazos y trazas
Arquitectura, datos e interfaces	`app/web.py`, `app/engine.py`, `app/camera.py`, `templates/`, `static/`, `docs/AVANCE5_DISENO.md`	Revisión técnica y trazabilidad código–requisito	Comprobar que el diseño representa la solución construida
Comportamiento del software	`tests/`, `docs/VALIDACION.md`, `docs/SIMULADOR.md`	Casos de prueba, pruebas de API y navegador	Verificar transiciones, persistencia, permisos, sincronización y recuperación
Calidad del reconocimiento	Imágenes propias por capturar y salidas de `tools/train_model.py` y `tools/evaluate_model.py`	Construcción de dataset por sesiones, entrenamiento y evaluación con conjunto de prueba independiente	Obtener matriz de confusión, exactitud y falsos positivos sin inventar valores
Operación integral	Cámara, herramientas, computador y tablet reales	Protocolo de diez ciclos, registro de incidencias y exportación del historial	Valorar el procedimiento completo y sus limitaciones

Situación de la evidencia contextual: el problema y los actores descritos en `docs/AVANCE4_ANALISIS.md` proceden de la formulación y del alcance técnico. No se han documentado en este apartado observaciones, entrevistas, registros institucionales ni hallazgos de campo. Antes de presentarlos como tales, el equipo debe identificar el laboratorio, obtener autorización cuando corresponda, aplicar el instrumento I-01 y conservar la evidencia.

El análisis combinará síntesis cualitativa de requisitos e incidencias con medidas descriptivas de las pruebas. No se harán inferencias sobre toda una institución a partir de un martillo ni se convertirá una puntuación de confianza en una afirmación de exactitud.

2. Objetivos que orientan la metodología

Se adoptan los objetivos de la base común `docs/avances/BASE_PROYECTO.md` y se asignan identificadores para facilitar la trazabilidad.

Objetivo general (OG). Desarrollar un prototipo local de préstamo de una herramienta que aplique autómatas finitos para controlar operaciones, conservar registros y verificar retiros y devoluciones con evidencia visual.

- **OE1.** Modelar inventario y operación mediante estados, eventos y transiciones, y definir un reconocedor de trazas que permita explicar ciclos válidos e inválidos.
- **OE2.** Implementar el motor, el almacenamiento SQLite y las interfaces de control y monitor, preservando la unicidad de préstamos activos y el rechazo de acciones inválidas.
- **OE3.** Preparar un dataset propio de cinco clases, ajustar un clasificador y evaluar sus resultados con datos separados del entrenamiento.
- **OE4.** Validar la lógica mediante pruebas automatizadas y completar diez ciclos documentados con cámara real y hardware simulado.

La operación modelada utiliza en el alcance actual señales simuladas de puerta y presencia. El OE4 comprende dos evidencias complementarias: las pruebas automatizadas ya documentadas y los diez ciclos con cámara real que siguen pendientes; los diez ciclos automatizados con observaciones controladas no satisfacen esta segunda parte.

3. Fases metodológicas

Resumen de fases y productos

Fase: 1. Análisis del problema	
Propósito	Delimitar actores, proceso, requisitos y alcance
Productos y evidencias	Descripción del problema, RF01–RF10, RNF01–RNF07, alcance e instrumento de contexto
Objetivos	OG, OE1
Estado frente al repositorio	Análisis técnico implementado; validación contextual pendiente
Fase: 2. Diseño de la solución	
Propósito	Formalizar reglas, arquitectura, datos e interfaces
Productos y evidencias	Dos autómatas, AFD reconocedor, arquitectura, modelo de datos y diseño de vistas
Objetivos	OE1, OE2
Estado frente al repositorio	Diseño y definiciones implementados; revisión académica y contextual pendiente
Fase: 3. Implementación o construcción	

Propósito	Construir el prototipo y habilitar la preparación del reconocimiento
Productos y evidencias	Motor, SQLite, portal, monitor, simulador, scripts de dataset/entrenamiento; posteriormente dataset y modelo propios
Objetivos	OE1, OE2, OE3
Estado frente al repositorio	Software implementado; dataset, entrenamiento y modelo pendientes

Fase: 4. Pruebas y validación

Propósito	Comparar comportamiento esperado y obtenido en niveles separados
Productos y evidencias	Casos automatizados, evaluación visual, diez ciclos reales, incidencias y ajustes
Objetivos	OE1–OE4
Estado frente al repositorio	Pruebas de software documentadas; validación visual y física pendiente

Fase: 5. Documentación y presentación

Propósito	Integrar método, resultados verificables, límites y sustentación
Productos y evidencias	Informe, metodología, plan de trabajo, presentación, anexos y demostración
Objetivos	OG, OE1–OE4
Estado frente al repositorio	Existen borradores y artefactos; integración final y evidencias reales pendientes

Fase 1. Análisis del problema

Propósito. Comprender y delimitar el préstamo antes de convertirlo en reglas de software, sin asumir que la descripción técnica ya fue validada por una institución.

Actividades.

1. Describir la secuencia actual de solicitud, entrega, retorno, revisión y registro en el laboratorio seleccionado.
2. Identificar a usuario, encargado y operador; confirmar quién decide disponibilidad, daño y reparación.
3. Registrar entradas —código de demostración, herramienta, observación visual, puerta y presencia—, procesos, salidas y excepciones.
4. Revisar RF01–RF10 y RNF01–RNF07 de `docs/AVANCE4_ANALISIS.md`, manteniendo fuera del alcance la multiplicidad de unidades, la autenticación productiva y el mecanismo físico.
5. Establecer criterios de aceptación por requisito y relacionarlos con un instrumento de validación.

Técnicas. Análisis documental, observación estructurada pendiente, modelado de proceso y matriz de requisitos.

Productos. Contexto validado por el equipo, requisitos revisados, alcance explícito y matriz I-02.

Criterio de cierre. Cada requisito tiene fuente, actor, criterio verificable y evidencia prevista; las afirmaciones contextuales cuentan con respaldo o se marcan como supuestos por confirmar.

Fase 2. Diseño de la solución

Propósito. Traducir las reglas del préstamo a modelos deterministas y a una arquitectura con una única autoridad de estado.

Actividades.

1. Definir el autómata de inventario con estados `Disponible`, `Reservada`, `Prestada`, `En revisión` y `Fuera de servicio`, sus siete eventos y las transiciones documentadas en `docs/INFORME.md`.
2. Definir el autómata de operación con los estados `Inactiva`, `Abriendo`, `Esperando retiro`, `Esperando depósito`, `Cerrando`, `Verificando`, `Completada` e `Incidencia`.
3. Especificar el AFD reconocedor con estado absorbente `Error` y aceptación de trazas válidas que terminan en `Disponible`; diferenciarlo del controlador interactivo, que rechaza una acción sin introducirla en una traza aceptada.
4. Diseñar la separación entre cámara, motor, adaptador de hardware, SQLite, control local y monitor de lectura.
5. Diseñar las tablas `users`, `loans`, `events`, `state` y `requests`, la restricción de un préstamo activo por herramienta y la idempotencia de solicitudes.
6. Preparar protocolos de prueba para rutas nominales, cancelación, objeto incorrecto, pérdida de evidencia, desconexión, daño y recuperación.

Técnicas. Tablas y diagramas de transición, lenguajes regulares, diseño modular, modelo de datos y diseño de casos de prueba.

Productos implementados. Definición compartida en `app/automata.py`; modelo y arquitectura en `docs/INFORME.md` y `docs/AVANCE5_DISENO.md`.

Criterio de cierre. Para cada par estado/evento definido existe como máximo un siguiente estado; las entradas no definidas se rechazan o conducen a `Error` en el reconocedor; cada requisito diseñado tiene un componente y una prueba asociados.

Fase 3. Implementación o construcción

Propósito. Materializar el diseño en un prototipo local reproducible y habilitar, sin dar por resuelto, el reconocimiento con herramientas propias.

Actividades de software implementadas.

1. Implementar transiciones, validación de propietario y rol, evidencia reciente/estable, incidencias y reintento en `app/engine.py`.
2. Confirmar estado, préstamo e historial mediante transacciones SQLite; impedir préstamos activos duplicados y repetición de solicitudes.
3. Implementar captura OpenCV, recorte de región de interés, conservación de la herramienta completa e inferencia YOLO11n-cls cuando exista un modelo compatible en `app/camera.py`.
4. Construir `/control`, `/monitor`, historial CSV, restricciones de escritura y aviso de desactualización en `app/web.py`, `templates/portal.html` y `static/`.
5. Construir el módulo independiente de `simulator/`, con escena virtual, recorrido manual y cinco escenarios automáticos, compartiendo las definiciones de `app/automata.py` pero no la base del módulo principal.
6. Incorporar `tools/train_model.py` y `tools/evaluate_model.py` para verificar estructura, entrenar y evaluar exclusivamente con imágenes propias.

Actividades propuestas y pendientes.

1. Configurar cámara y región de interés en una escena estable, conforme a `docs/RECONOCIMIENTO.md`.

2. Capturar las clases exactas `martillo`, `destornillador`, `alicate`, `vacio` y `otros`, con sesiones separadas para `train`, `val` y `test`. La meta orientativa existente es de aproximadamente 100 imágenes por clase, distribuidas cerca de 60/20/20; es una meta de construcción, no un resultado alcanzado ni una garantía de calidad.
3. Revisar manualmente etiquetas, objetos tapados, variación de orientación e iluminación y ausencia de pistas artificiales de fondo.
4. Ejecutar la comprobación de estructura y, solo si es satisfactoria, entrenar el modelo. Conservar `models/entrenamiento.json` y las salidas reales del entrenamiento.
5. Evaluar con `test` sin usarlo para ajustar repetidamente el umbral. Si cambia la cámara, la región o la futura caja física, capturar nuevas sesiones y repetir la validación pertinente.

Técnicas. Programación Python/Flask, transacciones SQLite, control de concurrencia, pruebas de API, captura y etiquetado por sesiones, ajuste de clasificador y control de configuración.

Criterio de cierre. El software permanece reproducible; el dataset pasa revisión estructural y visual; el modelo contiene exactamente las cinco clases y su evaluación queda archivada. Hasta que exista un modelo válido, el bloqueo de préstamos que exigen evidencia visual es el comportamiento esperado, no una falla que deba ocultarse.

Fase 4. Pruebas y validación

Propósito. Determinar qué afirmaciones admite cada nivel de evidencia y corregir defectos sin sustituir pruebas reales por simulaciones.

Estrategia por niveles.

1. **Pruebas formales y de motor.** Verificar transiciones, trazas, rechazo sin cambio de estado, evidencia posterior al cierre, cancelación, responsable, revisión, daño, reparación y recuperación.
2. **Pruebas de persistencia y API.** Comprobar préstamo único, idempotencia, transacciones, historial, permisos de escritura y ausencia de un endpoint público para inyectar reconocimiento.
3. **Pruebas de interfaz.** Ejercitar control y monitor, sincronización, reconexión y diseños adaptables; diferenciar dos páginas en un computador de una tablet conectada por la red real.
4. **Validación del dataset y del modelo.** Revisar separación por sesiones, imágenes legibles y etiquetas; calcular matriz de confusión, exactitud y fotogramas no martillo que superarían la aceptación individual configurada. Analizar también estabilidad temporal y errores operativos.
5. **Validación integral.** Ejecutar diez ciclos de préstamo/devolución con cámara real según `docs/DEMOSTRACION.md`, registrar cada intento y exportar el historial. Incluir por separado los casos de rechazo e incidencia; un intento fallido no se borra del registro para mejorar los resultados.

Resultados ya documentados, no recalculados aquí. La base común `docs/avances/BASE_PROYECTO.md` registra una ejecución del 5 de noviembre de 2026 con **47 pruebas aprobadas en 0,62 s**, mediante `.venv/bin/python -m pytest -q --basetemp=/tmp/avances-pytest-20261105`. Esta redacción no atribuye esa ejecución a la elaboración de la metodología. Como antecedentes, `docs/VALIDACION.md` registra 36 pruebas del módulo original y `docs/SIMULADOR.md` registra 47 en la suite conjunta tras añadir el simulador, incluidas 11 de ese módulo; también documenta un ciclo en navegador y una actualización local observada de 96 ms que no se repitió en la revisión de noviembre. Estas cifras describen ejecuciones específicas con observaciones controladas; no representan exactitud de visión, desempeño en la red de exposición ni validación con cámara real.

Criterio de cierre. Están archivadas las salidas de software; el modelo se evalúa sobre datos independientes; se ejecutan y registran diez ciclos reales con trazabilidad de préstamos y devoluciones; toda desviación tiene incidencia, análisis y decisión de ajuste o limitación. No se fija una exactitud mínima no sustentada: el equipo deberá acordar cualquier umbral de aceptación antes de la evaluación final y justificarlo según el riesgo de aceptar una herramienta incorrecta.

Fase 5. Documentación y presentación

Propósito. Comunicar el proceso y los resultados con correspondencia entre objetivos, método, evidencias y límites.

Actividades.

1. Integrar metodología y plan de trabajo con el análisis, el diseño, el informe y la presentación, sin cambiar nombres de estados, clases ni requisitos.
2. Anexar requisitos revisados, diagramas, casos de prueba, salidas de evaluación, historial CSV y capturas legibles, identificando su procedencia.
3. Separar en la sustentación lo implementado, lo documentado como ejecutado y lo pendiente; explicar expresamente la simulación de puerta y sensores.
4. Preparar una demostración acorde con las evidencias disponibles. Si no existe modelo entrenado, utilizar el simulador para explicar la lógica y no presentarla como reconocimiento real.
5. Realizar revisión cruzada de redacción, cifras, enlaces, autoría, declaración de IA y formato institucional antes de entregar.

Técnicas. Matriz de trazabilidad, revisión por pares del equipo, organización de evidencias y guion de sustentación.

Criterio de cierre. Cada afirmación cuantitativa tiene un archivo de respaldo; cada objetivo informa su estado y evidencia; no quedan marcadores sin resolver ni resultados propuestos redactados como logros.

4. Técnicas, herramientas y recursos

Técnica o recurso	Aplicación concreta	Sustento en el proyecto
Análisis documental	Contrastar alcance, objetivos, requisitos, diseño y límites	`README.md` y `docs/`
Observación estructurada	Validar el proceso real y las condiciones de operación; pendiente de aplicar	Instrumento I-01; contexto de `docs/AVANCE4_ANALISIS.md`
Autómatas finitos y trazas	Formalizar inventario, operación, rechazo y aceptación	`app/automata.py`, `docs/INFORME.md`
Matriz de trazabilidad	Relacionar objetivo, requisito, componente, prueba y evidencia	Instrumentos I-02 e I-03
Python, Flask y Waitress	Implementar motor, API y servicio local	`app/`, `simulator/`, `requirements.txt`
SQLite	Persistir estado, usuarios, préstamos, eventos e idempotencia	`app/engine.py`, `data/prestamos.sqlite`
HTML, CSS, JavaScript y SVG	Construir control, monitor, historial y diagramas	`templates/`, `static/`
OpenCV	Capturar video, delimitar la región y preparar imágenes	`app/camera.py`
Ultralytics YOLO11n-cls	Ajustar y evaluar clasificación de cinco clases cuando exista dataset propio	`tools/train_model.py`, `tools/evaluate_model.py`
Pytest	Automatizar motor, API, dataset y simulador	`tests/`, `README.md`
Playwright	Verificar interacción, sincronización y vistas en navegador	`tests/browser_*.cjs`, `tests/simulator_browser.cjs`

Técnica o recurso	Aplicación concreta	Sustento en el proyecto
Hoja de cálculo o CSV	Consolidar conteos, incidencias, latencias e historial sin alterar los archivos originales	Salida `/api/historial.csv`, `artifacts/predicciones.csv` cuando se genere
Git y documentación Markdown	Conservar cambios y facilitar revisión; el estado actual del historial debe comprobarse sin atribuir commits inexistentes	Repositorio y `docs/ESTADO.md`
Computador, cámara/celular, herramientas y tablet	Ejecutar validación pendiente en red local	`docs/RECONOCIMIENTO.md`, `docs/DEMOSTRACION.md`

La hoja de cálculo es un recurso de análisis propuesto, no una fuente de resultados existente. Las dependencias y comandos de ejecución se toman de `README.md`; no se presupone hardware que el código no implementa.

5. Relación entre objetivos, actividades y evidencias

Objetivo: OE1

Actividades decisivas	Fase 2: modelar estados, eventos, AFD y trazas; fase 4: probar transiciones
Indicadores o criterios	Cobertura de transiciones y rechazos; correspondencia entre evento registrado y transición definida
Evidencia disponible	`app/automata.py`, `docs/INFORME.md`, `tests/test_engine.py`, pruebas documentadas
Evidencia todavía requerida	Revisión académica final y consolidación de trazas en anexos

Objetivo: OE2

Actividades decisivas	Fase 3: implementar motor, persistencia, portal y monitor; fase 4: probar API y navegador
Indicadores o criterios	Un préstamo activo por herramienta; operaciones idempotentes; escritura local autorizada; monitor sin escritura; recuperación sin perder préstamos
Evidencia disponible	`app/engine.py`, `app/web.py`, `templates/`, `static/`, `tests/`, `docs/VALIDACION.md`
Evidencia todavía requerida	Confirmación de la versión que se entregará y prueba en la tablet/red reales

Objetivo: OE3

Actividades decisivas	Fase 3: capturar por sesiones, revisar, entrenar; fase 4: evaluar
Indicadores o criterios	Conteo por clase/conjunto/sesión; independencia de `test`; matriz de confusión, exactitud y falsos positivos; clases compatibles
Evidencia	Código de captura, entrenamiento y evaluación

disponible	
Evidencia todavía requerida	Dataset propio, `models/herramientas.pt`, `models/entrenamiento.json`, `artifacts/evaluacion_modelo.json` y `artifacts/predicciones.csv`

Objetivo: OE4

Actividades decisivas	Fase 4: ejecutar protocolo integral y analizar incidencias
Indicadores o criterios	Diez ciclos reales registrados; préstamo y devolución coherentes; ausencia de duplicados; evidencia posterior al cierre; latencias e incidencias descritas
Evidencia disponible	Guion y pruebas automatizadas de apoyo
Evidencia todavía requerida	Registro I-06 diligenciado, historial CSV, capturas reales y análisis de resultados

Objetivo: OG

Actividades decisivas	Integrar las cinco fases y sustentar decisiones
Indicadores o criterios	Trazabilidad completa y distinción entre lógica, visión y hardware
Evidencia disponible	Prototipo, informe y borradores
Evidencia todavía requerida	Contexto validado, cierre de OE3/OE4, revisión final y declaraciones institucionales

6. Instrumentos y procedimiento de validación

I-01. Guion de observación del proceso actual — pendiente de aplicación

Campos mínimos: laboratorio identificado por el equipo; fecha y duración que se registren al aplicarlo; observador por rol; autorización; paso observado; actor; entrada; registro utilizado; decisión; excepción; evidencia autorizada; diferencia frente al proceso propuesto.

Uso: observar solicitud, entrega, devolución y revisión sin inferir prácticas no vistas. El equipo decidirá si complementa la observación con documentos o entrevista; cualquier instrumento adicional deberá anexarse con su aplicación real.

Cierre: actores y reglas de RF01–RF10 revisados; discrepancias convertidas en cambio de requisito o limitación explícita.

I-02. Matriz de trazabilidad de requisitos

Campo	Contenido que debe registrarse
Identificador	RF/RNF y objetivo relacionado
Fuente	Documento, observación real o decisión técnica identificable
Componente	Archivo, función, tabla o vista responsable

Campo	Contenido que debe registrarse
Caso de prueba	Identificador, precondiciones, datos y resultado esperado
Resultado	Obtenido al ejecutar, con estado y salida asociada
Evidencia	Ruta del archivo o anexo; no una descripción sin respaldo
Decisión	Aceptar, corregir y repetir, o declarar limitación

I-03. Registro de casos de prueba de software

Campos mínimos: ID; requisito; versión o revisión del código; entorno; precondiciones; datos controlados; pasos; resultado esperado; resultado obtenido; estado; incidencia; responsable por rol; evidencia.

El catálogo inicial se deriva de `tests/`: rutas completas, evento inválido, evidencia antigua o repetida, baja confianza, objeto incorrecto, concurrencia, idempotencia, permisos, recuperación, dataset y escenarios del simulador. Las observaciones controladas se etiquetan como tales.

Comandos de referencia, no ejecutados para redactar este documento:

```
.venv/bin/python -m pytest -q --basetemp=.pytest_tmp
node --check static/app.js
node tests/browser_cycle.cjs
node tests/simulator_browser.cjs
```

La ejecución para la entrega debe registrarse con su salida real y entorno. No se reutiliza automáticamente el resultado histórico de `docs/VALIDACION.md`.

I-04. Ficha de control del dataset

Dimensión	Registro requerido
Procedencia	Herramienta/escena, clase, sesión y conjunto
Cantidad	Imágenes por clase y por `train`, `val`, `test`, calculadas desde los archivos
Independencia	Sesión exclusiva de un conjunto y ausencia de fotogramas casi idénticos repartidos entre conjuntos
Calidad	Legibilidad, etiqueta correcta, encuadre, orientación, escala, iluminación, oclusión y diversidad de `otros`/`vacio`
Riesgos	Fondos o textos que revelen la clase, datos personales, duplicados o cambios de cámara/ROI
Decisión	Aceptar para entrenar, corregir captura o repetir revisión

La comprobación automática se realiza con `.venv/bin/python tools/train_model.py --check`; la revisión visual es humana y no la sustituye el script.

I-05. Informe de evaluación del modelo

Se completa únicamente después de entrenar y ejecutar `.venv/bin/python tools/evaluate_model.py`. Debe conservar configuración, conteos, umbral utilizado, matriz de confusión, exactitud, falsos positivos de martillo, predicciones CSV y análisis de errores.

La exactitud se calcula como el número de predicciones correctas dividido por el total de imágenes evaluadas. Los falsos positivos de martillo del script son fotogramas cuya clase real no es martillo pero cuya predicción y puntuación superarían la aceptación individual configurada. Ninguna métrica de imágenes aisladas demuestra por sí sola estabilidad temporal, seguridad física o ausencia de engaño.

I-06. Protocolo y matriz de diez ciclos con cámara real

Cada ciclo comprende préstamo, devolución y aprobación siguiendo `docs/DEMOSTRACION.md`. Se registra una fila por ciclo, sin completar resultados anticipadamente:

Campo	Registro
Identificación	Ciclo 1–10, ejecución, operador y observador por rol
Condiciones	Cámara, herramienta, configuración/ROI, modelo, iluminación y red
Préstamo	Inicio/fin registrados al ejecutar; estados finales; préstamo creado; incidencias
Devolución	Inicio/fin registrados al ejecutar; evidencia posterior al cierre; cierre del préstamo; aprobación
Integridad	Un préstamo por ciclo, responsable correcto, sin duplicados ni omisión de devolución
Sincronización	Momento de cambio y observación en monitor si se mide; método de medición
Resultado	Completo, rechazado o con incidencia; causa y recuperación
Evidencia	Historial CSV, capturas y anotaciones asociadas

Se informan ciclos completos, tipos de fallo y, si se mide de forma consistente, latencia mediante valores descriptivos y su método. No se descartan fallos del denominador ni se mezclan con los diez ciclos del motor automatizado.

I-07. Lista de revisión académica final

- Objetivos, fases, requisitos y diseño usan la misma terminología.
- Toda cifra tiene fuente, fecha de ejecución registrada en su evidencia y alcance.
- Las pruebas simuladas no se presentan como campo, cámara real o custodia física.
- Los resultados pendientes permanecen marcados y no se reemplazan por valores de ejemplo.
- Las imágenes y datos usados cuentan con autorización y minimización de datos personales.
- Se revisan declaración de IA, aportes del equipo, referencias, anexos y formato oficial.

7. Gestión de datos, ética y limitaciones

Las imágenes deben centrarse en herramientas y en la región de interés. Se evitarán rostros, identificaciones y datos personales; si aparecen de forma incidental, el equipo deberá excluirlos o tratarlos conforme a la autorización institucional. No se transferirán imágenes a servicios externos sin decisión informada del equipo. La alternativa de Colab mencionada en `docs/RECONOCIMIENTO.md` es opcional y no está activada.

Los códigos `1001`, `1002` y `ADMIN` son datos de demostración, no autenticación productiva ni identidades reales. El control local con origen y token reduce escrituras no autorizadas en el alcance del prototipo, pero no constituye una evaluación integral de seguridad.

La clasificación reconoce apariencia de categorías, no identidad de una unidad ni daños. La revisión de daños es humana. Una imagen congelada o un engaño visual no quedan descartados por esta versión, y una señal de presencia no

demuestra custodia. Los resultados obtenidos con una escena no se generalizan automáticamente a otra cámara, iluminación, caja o institución.

8. Declaración de uso de inteligencia artificial

Para la actualización de `docs/AVANCE3_METODOLOGIA.md` y `docs/avances/PLAN_TRABAJO.md` se utilizó **Kimi Code CLI** como apoyo para revisar la guía del Avance 3, organizar la estructura académica, redactar y contrastar afirmaciones con archivos del repositorio. La herramienta no inventó entrevistas, observaciones de campo, nombres, fechas de ejecución, mediciones ni resultados experimentales; los apartados sin evidencia se formularon como actividades pendientes o propuestas.

La verificación documental se apoyó en `README.md`, el código de `app/`, `simulator/` y `tools/`, las pruebas de `tests/` y los documentos técnicos existentes. Las ejecuciones citadas proceden de `docs/avances/BASE_PROYECTO.md`, `docs/VALIDACION.md` y `docs/SIMULADOR.md`; esta actualización no afirma haberlas vuelto a ejecutar.

El equipo conserva la responsabilidad de comprender las decisiones, revisar y editar el contenido, ejecutar las validaciones, completar los aportes individuales y adaptar esta declaración a las reglas de la docente y de la institución. También debe declarar cualquier otro uso de IA no cubierto por este apartado. No se ingresaron intencionalmente datos personales, confidenciales o sensibles para elaborar estos documentos.

9. Condiciones para continuar

La secuencia operativa, los responsables por rol, las dependencias y las evidencias se detallan en `docs/avances/PLAN_TRABAJO.md`. Antes de dar por validado el proyecto, el equipo debe asignar personas a los roles, convertir las semanas relativas al calendario oficial, validar el contexto, crear y evaluar el dataset/modelo, ejecutar los diez ciclos reales y probar la tablet en la red de demostración. La construcción de Arduino, puerta y sensores físicos pertenece a una etapa posterior y exigiría una metodología y validación adicionales.